

CA Glue Filling Station

Ladder Logic Program & Learning Guide — every principle and algorithm explained for programming, operation and troubleshooting

Platform	CODESYS 3.5 — IEC 61131-3, language LD (ladder diagram)
Program	PLC_PRG — 58 networks, 10 sections (A–J)
Packaging modes	Squeeze bottle and syringe, selected by fixture sensor
Start principle	Two-hand opto buttons (0.5 s synchronism), auto-stop at high level
Companion files	GVL_IO.txt (variables) · PLC_PRG_Ladder.txt (rung listing) · PLC_PRG.st (ST mirror)
Revision	A — 2026-08-05

Read this first — safety boundary

The E-stop, two-hand control and guard-door lock in this program are SEQUENCE CONTROL AND INDICATION ONLY. The protective functions themselves must be implemented in safety-rated hardware (safety relay or safety PLC, selected per an ISO 13849-1 risk assessment) that removes valve power independently of this program. Chapter 2 explains exactly where that boundary lies.

How to use this document. Chapter 1 explains what the machine does. Chapters 3–4 give you the vocabulary (I/O map, ladder elements). Chapter 5 is the heart: every network of the program, drawn as a ladder diagram, with the principle behind it and what to check when it misbehaves. Chapters 6–7 turn that knowledge into a troubleshooting routine and a commissioning checklist.

Contents

- 1 · System overview — what the station does
- 2 · Safety architecture — what the PLC does and does not do
- 3 · I/O reference — every signal, its wiring and its purpose
- 4 · Ladder-logic fundamentals — the building blocks used in this program
- 5 · The program, network by network (sections A–J)
- 6 · Troubleshooting guide — fault codes, symptoms, online diagnosis
- 7 · Commissioning & test checklist
- Appendix A · Variable cross-reference (auto-generated from the program)
- Appendix B · Importing the program into CODESYS

1 · System overview

The station fills cyanoacrylate (CA) glue from a pressurised pot into one of two package types: a **squeeze bottle** or a **syringe**. A fixture sensor tells the program which fill setup is installed, so the operator never selects a mode manually — the machine knows. The operator loads a package behind a guard door, closes the door and presses two opto-touch buttons at the same time. The program locks the door, opens the matching fill valve, and closes it automatically when the package's high-level sensor trips. A lamp tells the operator the package is full; opening the door and removing the package arms the next cycle.

1.1 · The operating sequence as a state machine

The whole program hangs on four sequence states plus a fault overlay. Every rung in Section G either moves the machine forward along this diagram or refuses to. If you remember one picture from this document, make it this one:

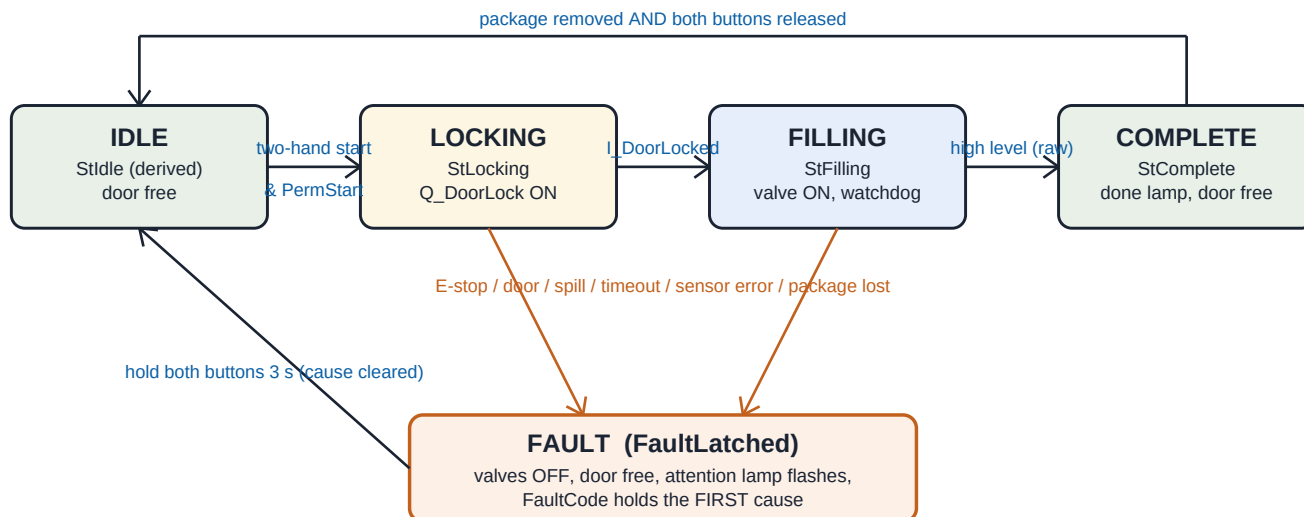


Figure 1 — the sequence state machine. Solid arrows are normal flow; the orange region is reachable from every cycle state and always de-energises both valves.

1.2 · A normal bottle fill, step by step

1. Operator installs the bottle fixture → I_BottleMode TRUE → bottle mode lamp ON (network 48).
2. Operator inserts an empty (or partly full) bottle → I_BottlePresent TRUE, debounced 100 ms (network 03).
3. Operator closes the door → I_DoorClosed TRUE. ReadyBottle and PermStart become TRUE (networks 22, 25).
4. Operator presses both opto buttons within 0.5 s → StartPulse (networks 10–14) → state LOCKING, door lock energised (networks 26, 47).
5. Lock feedback I_DoorLocked arrives (< 2 s) → state FILLING → bottle valve opens (networks 29, 45); filling lamp ON (network 50).
6. Glue rises past the low-level sensor (this is how the program distinguishes a first fill from a top-off) and reaches the high-level sensor → valve closes THE SAME SCAN → state COMPLETE, done lamp ON (networks 32, 51).
7. Operator releases the buttons, opens the now-unlocked door and removes the bottle → state IDLE, ready for the next package (network 33).

A syringe fill is identical except that the syringe fixture reports mode, the package permissives also demand the syringe be clamped (I_SyrLocked) and its fill tip present (I_TipPresent), and the fill ends on the plunger-high detector instead of a liquid level.

When anything goes wrong

Anything that interrupts this happy path — E-stop, door, spill, a sensor that stops making sense, a fill that takes too long — lands in the FAULT state with both valves closed, and **FaultCode** remembers the FIRST cause (chapter 6). The operator clears it by holding both opto buttons for 3 seconds after removing the cause.

2 · Safety architecture

A standard PLC executing this program is **not a safety device**: a single output transistor failure, a crashed runtime or a programming mistake could hold a valve open. Machine-safety standards therefore split the work in two layers:

Layer	Implemented in	Job
Protective layer	Safety relay / safety PLC + safety-rated devices (ISO 13849-1)	E-stop circuit, ISO 13851 two-hand relay, guard-lock interlock. Removes power from the fill-valve solenoids and the pressure supply independently of the standard PLC .
Sequence layer (this program)	CODESYS application, standard I/O	Decides WHEN a fill should happen, supervises timing, catches implausible sensors, drives lamps and counters, remembers the first fault cause. Mirrors the safety signals as ordinary inputs so it can annunciate and sequence around them.

In practice: the E-stop chain and two-hand relay outputs are wired both into the safety contactor feeding the valve solenoids *and* into the PLC inputs I_EStopOK / I_BtnLeft / I_BtnRight used here. When you troubleshoot 'valve will not open', remember there are two layers that can be blocking it — check the safety relay's LEDs as well as the program's permissive chain.

2.1 · Fail-safe wiring conventions

Every safety-relevant sensor in this design is wired so that the **healthy condition reads TRUE** (normally-closed contacts, light-on opto sensors). A cut cable, dead sensor or unplugged connector then looks like a problem and stops the machine, instead of silently looking like 'all clear'. This is the single most important wiring principle on the station:

Signal	Wired	TRUE means	A broken wire reads as
I_EStopOK	NC chain	E-stop released, circuit intact	E-stop pressed → machine stops
I_DoorClosed	NC in switch	door physically closed	door open → no start, fill aborts
I_DoorLocked	NC in lock	lock bolt engaged	unlocked → fill aborts
I_PotLevelOK	NC / light-on	glue level above minimum	pot empty → new fills blocked
I_SpillDetect	NO	liquid detected (event signal)	no detection — this sensor is supervised by routine inspection instead; it is an annunciation aid, not a protective device

2.2 · De-energise-to-safe outputs

Both fill valves must be **spring-return, normally-closed** valves: glue flows only while the solenoid is energised. Every abort path in this program works by making a coil FALSE — which only translates to 'glue stops' if the valve closes when power is removed. The door lock is the opposite kind of choice: it is energise-to-lock, so a power failure leaves the door openable (nobody is locked out during a blackout; the safety layer keeps the process safe).

Two-hand gesture vs two-hand protection

Section C implements a two-hand START GESTURE modelled on ISO 13851 concepts — synchronism window, tie-down lockout, full-release re-initiation — so the sequence also refuses discordant presses. It is **NOT** a two-hand protective function: the fill deliberately continues after the buttons are released, because during the fill the locked guard door protects the operator, not the buttons. Actual hand protection comes from the hardware two-hand relay in the protective layer, sized and positioned per the standard.

3 · I/O reference

Addresses are placeholders — remap them to your coupler in the CODESYS device editor. The 'watch when' column is the fastest route from a symptom to a signal worth watching in the online view.

3.1 · Inputs

Variable	Addr	Device	Watch when...
I_BtnLeft / I_BtnRight	%IX0.0 / .1	opto-touch buttons	starts refused: do both go TRUE within 0.5 s of each other?
I_DoorClosed	%IX0.2	guard-door switch	no start / fault 2: does it flicker when the door is bumped?
I_DoorLocked	%IX0.3	guard-lock feedback	fault 8: does it come TRUE within 2 s of the lock energising?
I_PotLevelOK	%IX0.4	pot level sensor	attention lamp flashes: pot genuinely low, or sensor fouled?
I_BottleMode	%IX0.5	fixture ID sensor	wrong mode lamp / fault 6: seated fixture, clean target?
I_BottleLow	%IX0.6	bottle low level	top-off behaviour odd / sensor error 19: alignment, residue
I_BottleHigh	%IX0.7	bottle high level	overflow or early stop: trips exactly at the fill line?
I_BottlePresent	%IX1.0	bottle presence	fault 7 / refuses start: detects bottle, not the nest wall?
I_TipPresent	%IX1.1	syringe tip sensor	syringe start refused: tip really seated?
I_PlungerLow	%IX1.2	plunger at bottom	sensor error 20: both plunger sensors TRUE = impossible
I_PlungerHigh	%IX1.3	plunger at top	syringe overflow / early stop
I_SyrPresent	%IX1.4	syringe presence	fault 7 / refuses start
I_SyrLocked	%IX1.5	clamp feedback	refuses start: clamp fully over-centre?
I_EStopOK	%IX1.6	E-stop chain (NC)	E-stop lamp ON: chain open somewhere — find which device
I_SpillDetect	%IX1.7	drip-tray probe	fault 3 repeats: real leak, or conductive residue on probe?

3.2 · Outputs

Variable	Addr	Device	Driven by
Q_ValveBottle	%QX0.1	bottle fill valve (NC, spring return)	network 45
Q_ValveSyringe	%QX0.0	syringe fill valve (NC, spring return)	network 46
Q_LampBottle	%QX0.2	packaging lamp — bottle	network 48
Q_LampSyringe	%QX0.3	packaging lamp — syringe	network 49
Q_LampFilling	%QX0.4	fill-valve-operating lamp	network 50
Q_LampDone	%QX0.5	package-filled lamp	network 51
Q_LampEStop	%QX0.6	E-stop condition lamp (steady)	network 52
Q_LampAttention	%QX0.7	operator-interaction lamp (flashes 1 Hz)	networks 53–54
Q_DoorLock	%QX1.0	guard-lock solenoid (energise-to-lock)	network 47

Design note — the added output

Q_DoorLock is not in the original output list — it was added because the input list names a door lock, and a lock needs a command signal. If your door uses a purely mechanical latch, delete network 47 and wire I_DoorLocked from the latch position switch instead.

3.3 · Diagnostic variables (no wiring — watch via HMI or online view)

Variable	Meaning
FaultCode	0 = none · 1 E-stop · 2 door · 3 spill · 4 fill timeout · 5 sensor plausibility · 6 mode lost · 7 package lost · 8 lock failed — always the FIRST cause since the last reset

Variable	Meaning
CntBottlesFilled / CntSyringesFilled	completed fills per package type (cmdCounterReset clears both)
StIdle...StComplete, CycBottle/CycSyringe	live state of the sequence machine — the first thing to check online
CycTopOff	TRUE while the running/last cycle started on a package that already held glue (bottle low level made / plunger off its empty stop) — explains a legitimately short fill

4 · Ladder-logic fundamentals used in this program

Everything in chapter 5 is built from seven ideas. If these are familiar, skip ahead; if a rung ever confuses you, come back here.

4.1 · The scan cycle — why order matters

A PLC does not execute rungs 'whenever'. Every few milliseconds it (1) copies all physical inputs into memory, (2) executes network 01, then 02, ... then 58 using that frozen snapshot, (3) copies the result memory to the physical outputs. Three practical consequences:

Inputs cannot change mid-scan. If network 03 and network 45 both look at I_BottlePresent, they see the same value even if the wire changed in between.

Later networks see earlier results from THIS scan; earlier networks see results from the LAST scan. That is why ResetPulse (network 15) is computed before every latch that consumes it (16, 18, 44) — the reset acts in the same scan. The one place this program deliberately accepts a one-scan-old value is tResetHold reading FaultLatched (network 15 reads what networks 35–42 wrote last scan) — irrelevant against a 3-second hold.

An output coil is just memory until the scan ends. Forcing a variable in CODESYS changes memory; the physical output follows at the end of the scan.

4.2 · Contacts are questions, coils are answers

A normally-open contact —|— asks 'is this variable TRUE?'; a normally-closed contact —|/— asks 'is it FALSE?'. Contacts in series are AND; parallel branches are OR. A coil —()— writes the result of the whole rung into a variable, every scan. **Do not confuse contact type with wiring type:** I_EStopOK is wired normally-closed electrically (chapter 2), yet most rungs ask —|— I_EStopOK because the PLC input is TRUE when healthy. The ladder symbol chooses the question, the wiring chooses the truth.

4.3 · Latches — making the PLC remember

A rung's coil forgets everything each scan, so memory must be built deliberately. This program shows both classic idioms on purpose:

Seal-in: the output variable appears as a contact in parallel with its own trigger, so once TRUE it keeps itself TRUE until a series condition breaks the seal. Used for THC_Lockout (12), EStopTrip (16), SpillTrip (18) — places where the RESET CONDITION is the interesting part of the story.

SET / RESET coils: —(S)— makes a variable TRUE and leaves it; —(R)— makes it FALSE. The variable holds its value between scans with no seal rung. Used for the state machine and FaultLatched (26–44), where different rungs own the setting and the clearing. Rule of thumb: when one rung both sets and clears, seal-in; when responsibility is split across rungs, S/R. If an S and an R fire in the same scan, the LAST executed network wins — network 43 (abort) is deliberately placed after every transition rung for exactly this reason.

4.4 · The TON timer — one block, four jobs

TON (on-delay) has two pins that matter: IN and Q. While IN is TRUE, elapsed time accumulates; Q turns TRUE when the preset PT is reached; the moment IN drops, everything resets instantly. From this one behaviour the program builds four different tools:

Job	Idea	Where
Debounce / qualify	signal must be steadily TRUE for PT before it is believed; dis-believed instantly when it drops (fail-safe asymmetry)	03–09, 17, 19–20
Discordance detector	times how long ONE button is held alone; Q = the 0.5 s window was missed	10–11
Watchdog	times how long a state is allowed to last; Q = something is stuck	28, 30–31
Gesture recogniser	times how long the operator deliberately holds both buttons	15

4.5 · Edge detection — R_TRIG

R_TRIG's Q is TRUE for exactly one scan when its CLK input goes FALSE→TRUE. It converts a condition ('both buttons are down') into an event ('both buttons JUST went down'). Every start in this machine is an event, never a level — that is what forces the operator to actuate the buttons freshly for every cycle (anti-repeat). Its falling-edge twin F_TRIG guards the other direction: network 12 uses it to catch the two-hand condition DROPPING while a button is still pressed, which is what enforces full-release re-initiation.

4.6 · The two-timer flasher

Networks 01–02 cross-couple two TONs: A times while B's Q is FALSE; when A finishes, B starts; when B finishes it resets A and everything repeats. Blink follows tBlinkA.Q, giving a square wave with period $2 \times \text{cfgBlinkHalfPeriod}$ (1 Hz). ANDing any condition with Blink (network 54) turns a steady condition into a flashing lamp — one oscillator serves every flashing indication you will ever add.

4.7 · Compare and move — carrying a number through ladder

Ladder is not only booleans. An EQ box passes power when two numbers are equal; a MOVE box copies a number when it receives power. Networks 35–42 chain them — 'if FaultCode is still 0, write MY code into it' — which is the whole first-out algorithm in two boxes (section H).

5 · The program, network by network

Each section below starts with the principle it implements, then shows its networks exactly as you will transcribe them into CODESYS. The italic line under each network is the note to remember when troubleshooting it. Network order is part of the design — keep it.

Section A · System utilities (networks 01–02)

Principle: **the two-timer oscillator**. Two on-delay timers reset each other in a loop, producing a free-running square wave (Blink) with period $2 \times \text{cfgBlinkHalfPeriod} = 1 \text{ s}$. Building the flasher once, at the very top of the program, means every 'flashing' indication is one AND away — and all flashing lamps blink in phase, which operators read as one coherent machine rather than several random blinkers.



Half of the oscillator: tBlinkA times only while tBlinkB.Q is FALSE. If Blink ever freezes, watch both timers' ET online — one of them is being held in reset.



The other half, plus the Blink coil. Period = $2 \times \text{cfgBlinkHalfPeriod}$. All flashing indications derive from this one bit, so they blink in phase.

Section B · Input conditioning (debounce) (networks 03–09)

Principle: **debounce and signal qualification**. Real sensors chatter: a bottle dropped into the nest bounces, an opto sensor flickers as a syringe slides past. Each raw input feeding a decision is therefore passed through a TON: the signal must hold TRUE for the debounce time before the program believes it. Note the deliberate asymmetry — trust is built slowly (100 ms) but withdrawn instantly, because for a permissive signal the safe direction of doubt is OFF. The db* variables, not the raw I_* inputs, are what the rest of the program consumes for presence decisions; the only raw uses are the fill-stop sensors (speed matters more than noise immunity) and the hard interlocks repeated in the valve rungs.



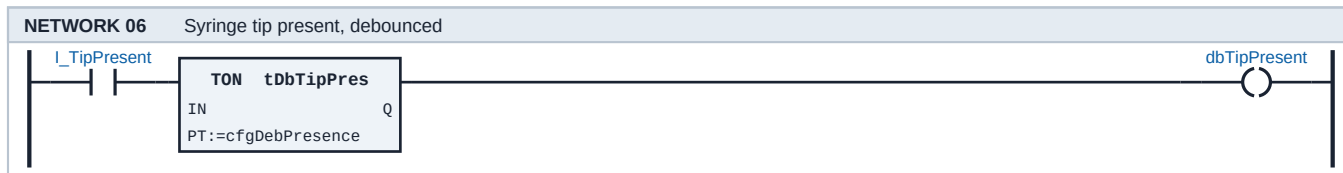
A bottle must sit still for 100 ms before dbBottlePresent is believed; removal is believed instantly. If starts are refused with a bottle loaded, watch this bit — chattering here means nest vibration or a marginal sensor gap.



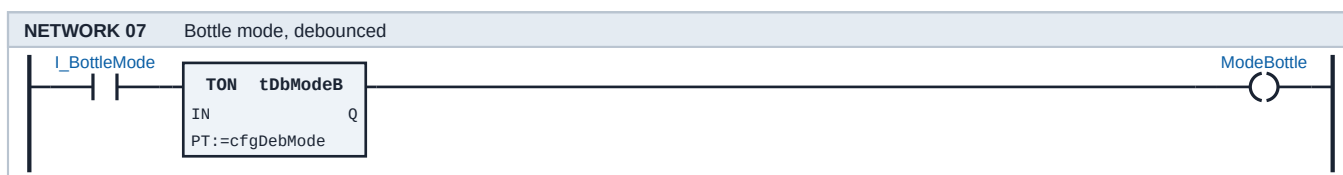
Same pattern for the syringe body sensor.



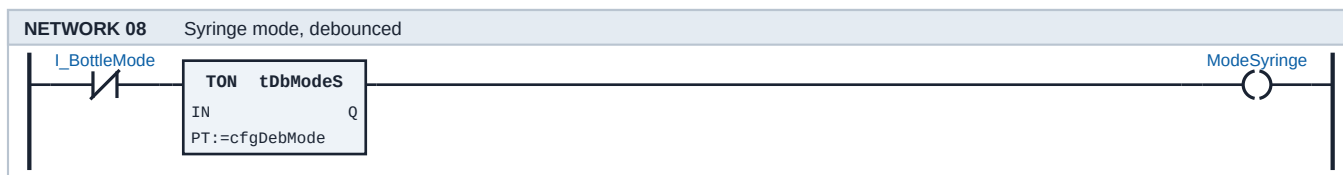
Same pattern for the clamp feedback. A clamp that is 'almost' over-centre gives a flickering *I_SyrLocked* — the debounce hides the flicker, the plausibility check (network 20) catches the contradiction if it drops mid-fill.



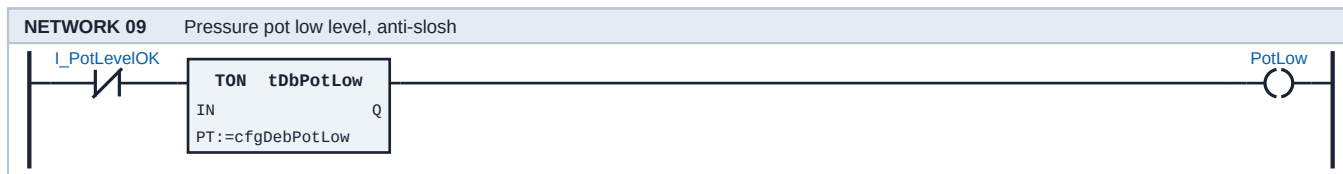
Same pattern for the fill-tip sensor.



Mode = bottle only after the fixture signal holds for 200 ms.



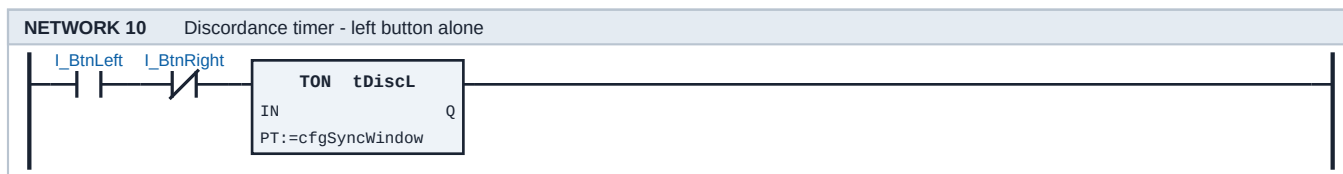
Mode = syringe is a separately debounced NOT of the same input — so during a fixture swap BOTH modes are FALSE and the station simply cannot start. 'Both mode lamps off' therefore means: signal changing, or sensor not seeing its target.



PotLow needs the level to read low for a full 2 s — sloshing from a bottle being slammed in cannot flash the refill warning. PotLow blocks NEW starts only (network 25); a fill in progress finishes on the glue in the line.

Section C · Two-hand control & reset gesture (networks 10–15)

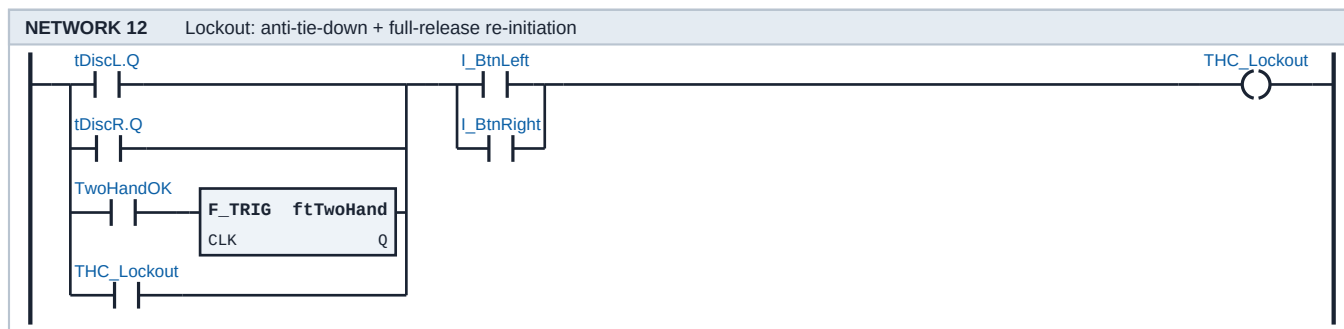
Principle: a **two-hand START gesture modelled on ISO 13851 concepts** — not a two-hand protective function (the fill deliberately continues after release, because during the fill the LOCKED GUARD protects the operator, not the buttons; the hardware two-hand relay in the safety layer provides the actual hand protection). Four properties are enforced: **synchronism** — both buttons within 0.5 s of each other; **anti-tie-down** — taping one button down and pressing the other is caught by the discordance timers and latched as a lockout that only full release clears; **full-release re-initiation** — if the two-hand condition drops with a button still pressed, the same lockout latches, so re-pressing one button can never re-form the condition; **anti-repeat** — the start signal is a rising EDGE. The same station doubles as the fault-reset input via a release-then-hold-3-s gesture, saving a dedicated reset button.



Times how long the LEFT button is held alone. Reaching 0.5 s means the press was discordant.



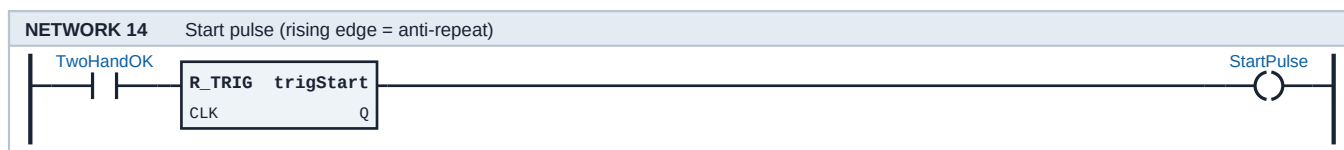
Mirror image for the right button.



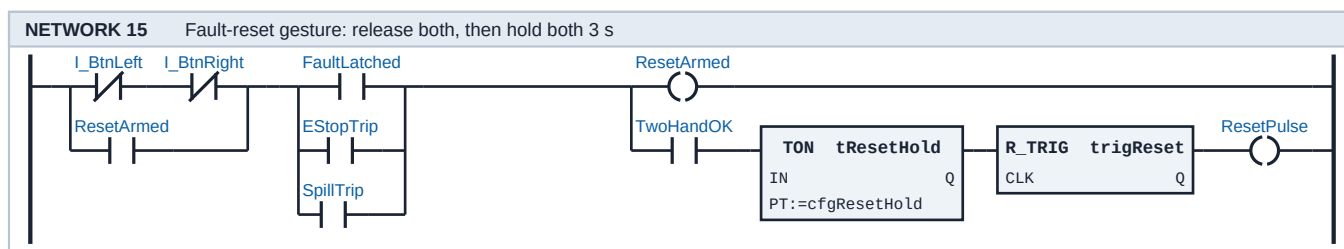
The lockout latch, with three ways in and one way out: either discordance timer (button held alone > 0.5 s), or the falling edge of TwoHandOK while a button is still pressed (F_TRIG — this is what forces a FULL release before re-initiation; F_TRIG reads last scan's TwoHandOK because network 13 runs after this one). The seal holds while any button is pressed, so ONLY releasing both clears it. Symptom: operator swears the buttons are dead — they are holding one down from the last attempt.



The two-hand condition itself: both pressed AND no lockout. Note this is a LEVEL — network 14 turns it into an event.



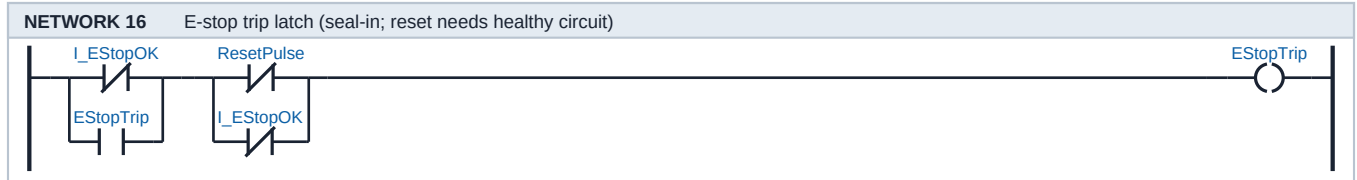
R_TRIG makes the start a one-scan pulse: anti-repeat for free. Combined with network 12's full-release rule, every StartPulse provably comes from both buttons freshly applied within 0.5 s.



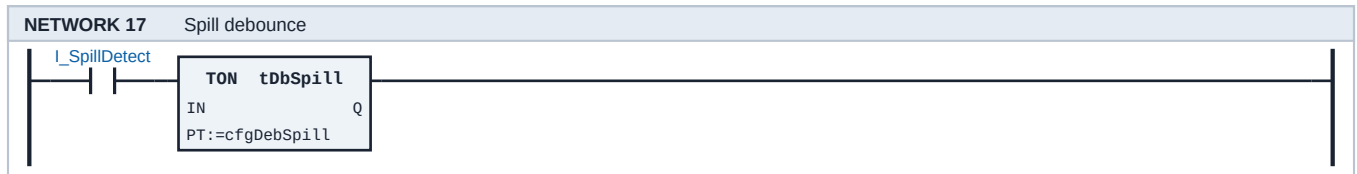
The reset gesture, in two steps that both matter: ResetArmed latches only when BOTH buttons are seen released while a trip is latched — hands resting on the buttons when a fault occurs can never auto-acknowledge it. Then holding both for 3 s fires the one-scan ResetPulse. Reading FaultLatched here uses last scan's value (it is computed in section H) — irrelevant across a 3 s hold.

Section D · Safety-event latches (networks 16–18)

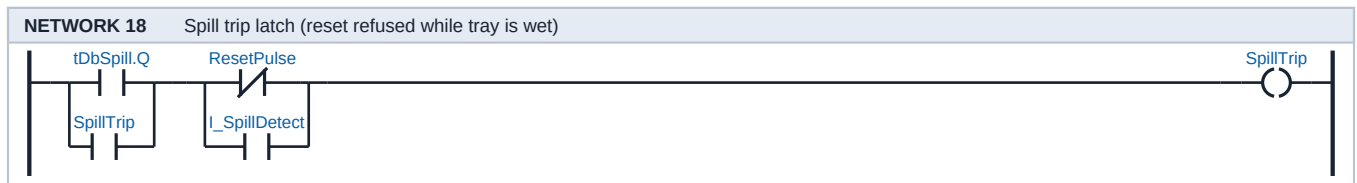
Principle: **trip latching**. A safety event must be remembered, not merely mirrored: if a momentary E-stop tap or one droplet across the spill probe merely paused the machine, it would resume by itself the moment the signal cleared — the operator would never know, and an intermittent fault would never be investigated. Both latches use the seal-in idiom, and both refuse to release until the reset gesture arrives AND their own cause is verifiably gone.



Seal-in latch, the textbook rung for the idiom: trigger branch (E-stop circuit open), seal branch (EStopTrip itself), and the seal-break term $\text{NOT}(\text{ResetPulse AND } I_E\text{StopOK})$, drawn by De Morgan as two parallel NC contacts. The latch survives everything except a reset gesture delivered while the circuit is healthy — and it trips even on a momentary tap.



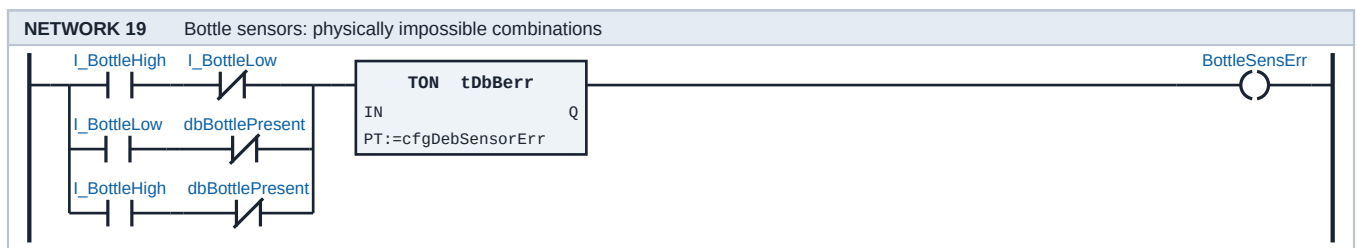
50 ms qualification for the spill probe — fast enough for a real leak, deaf to a single splash droplet bouncing past.



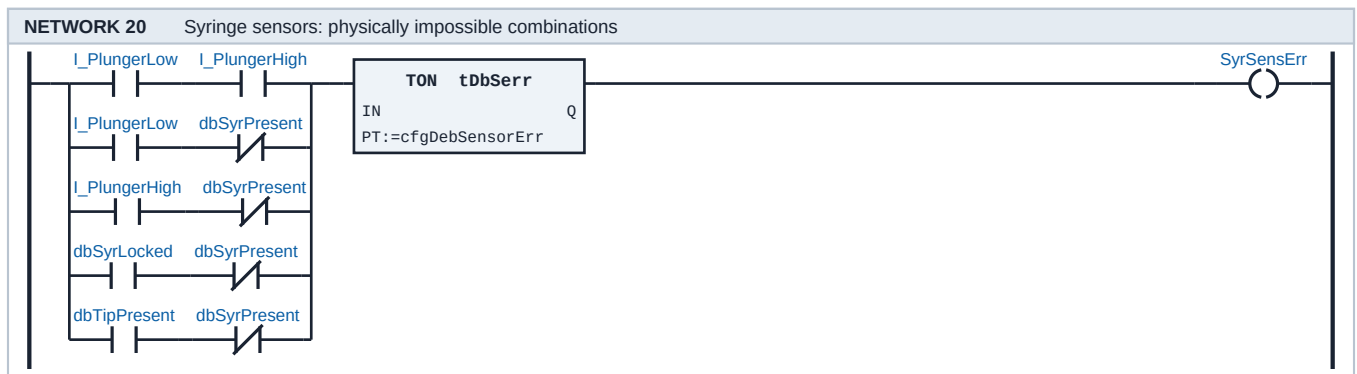
Same latch idiom as 16, one subtlety different: the seal-break requires the tray to read DRY, so the reset gesture is REFUSED while $I_SpillDetect$ is still TRUE. Clean first, then reset. Note the spill input's own limitation (see GVL): a dead probe reads 'dry' forever — proof-test it at shift start.

Section E · Sensor plausibility (false-positive defence) (networks 19–21)

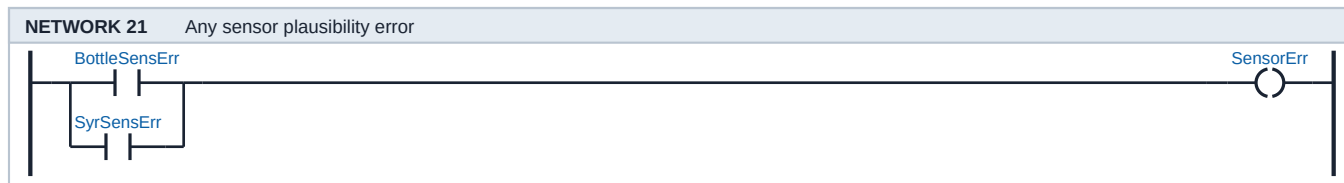
Principle: **plausibility checking** — the program's answer to 'how do I ensure a presence sensor isn't giving a false positive?'. One sensor can lie; a physically impossible COMBINATION of sensors cannot happen on a healthy machine. Glue rises bottom-up, so HIGH without LOW is impossible; a level reading with no bottle present is impossible; a plunger cannot be at both ends; a clamp cannot be locked on a syringe that is not there. Any such combination, held for 0.5 s, raises SensorErr — which blocks new starts, flashes the attention lamp, and (if it appears mid-cycle) aborts as fault 5. CA-specific tip: cyanoacrylate vapour fogs optical sensors; when SensorErr appears, clean lenses first.



Three impossible bottle situations: HIGH without LOW (glue rises bottom-up), and either level signal with no bottle in the nest. Held 0.5 s → BottleSensErr. When it fires, the offending branch is highlighted online — it points at the exact lying sensor.



Five impossible syringe situations: plunger at both ends; either plunger signal with no syringe in the nest (catches a stuck-ON plunger sensor); clamp locked with no syringe; fill tip seen with no syringe. Same 0.5 s confirmation as network 19.



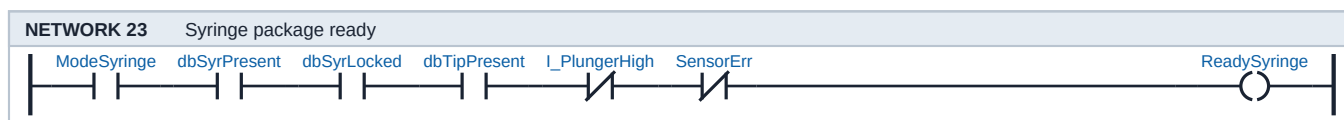
The OR of both checkers, consumed by the ready rungs (22–23), the attention lamp (53) and the mid-cycle fault (40).

Section F · Ready conditions & start permissive (networks 22–25)

Principle: **the permissive chain**. Every condition required for a safe, sensible start is a contact in series — the rung is a checklist the machine walks every scan. The payoff is diagnostic: online, the first cold contact in the chain IS the reason the machine refuses to start; no cross-referencing needed. Requirements are split by package type (ReadyBottle / ReadySyringe), common safety (SafetyOK), and cycle context (door, pot level, idle) so each rung stays short enough to read at a glance.



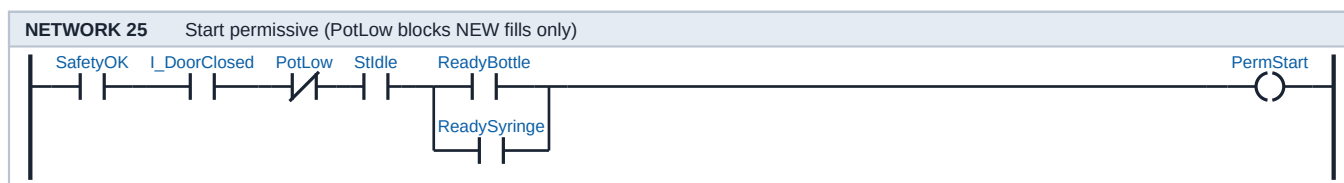
The bottle checklist: bottle mode, bottle present (debounced), NOT already full, sensors plausible. NOT I_BottleHigh is what refuses an already-full bottle; a partly-filled one (I_BottleLow TRUE) passes — that is a legal top-off, which network 27 records in CycTopOff.



The syringe checklist adds the clamp and the fill tip — a syringe that is present but not locked, or missing its tip, never starts.



Common safety summary used by network 25: E-stop healthy AND no latched trip of any kind.



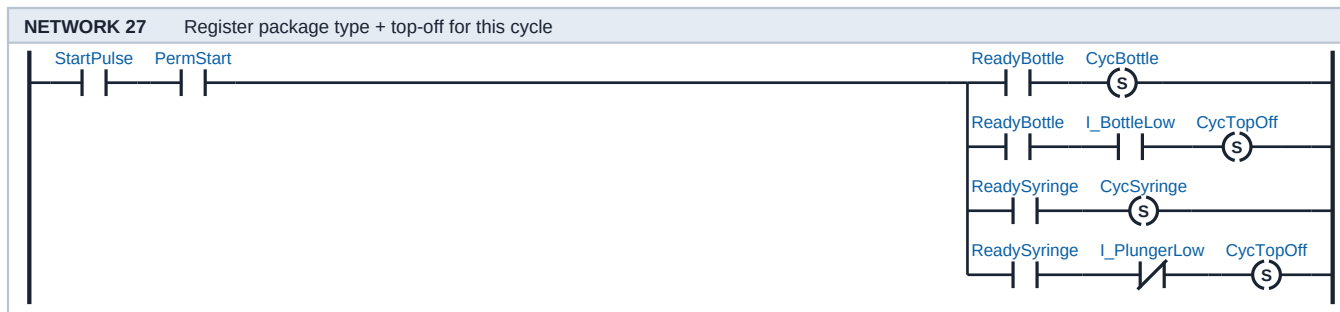
THE start permissive. Online, walk left to right: the first cold contact is the reason the machine refuses to start. PotLow appears here — and only here — so low glue blocks new fills without aborting a running one.

Section G · Sequence state machine (networks 26–34)

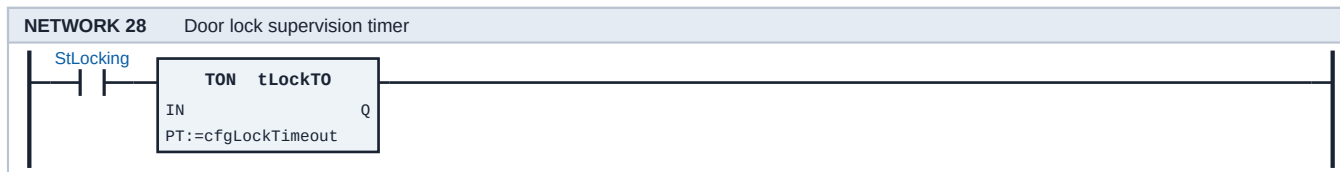
Principle: **the state machine**. Four mutually exclusive states carry the cycle: LOCKING → FILLING → COMPLETE, with IDLE derived as 'none of the above and no trip'. Each transition is one rung that SETs the next state and RESETs the current one, so the machine can never be in two cycle states at once, and every transition has exactly one place to put a breakpoint. A second idea rides along: **cycle registration** — the package type is captured into CycBottle/CycSyringe at the start edge, so a fixture sensor glitching mid-fill can never swap which valve is open (it aborts as fault 6 instead).



The only rung that starts a cycle: the one-scan StartPulse gated by the full permissive. SET coil — from here the state machine owns the bit.



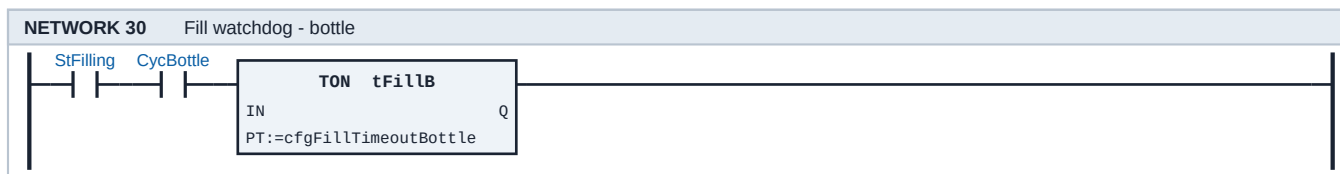
Cycle registration: which package type this cycle fills is CAPTURED now, from the same gated pulse — everything downstream consults CycBottle/CycSyringe, never the live mode sensor. CycTopOff additionally records that the package already held glue at start (bottle low level made / plunger off its empty stop): a top-off legitimately finishes much sooner than a full fill, and this bit says so on the HMI.



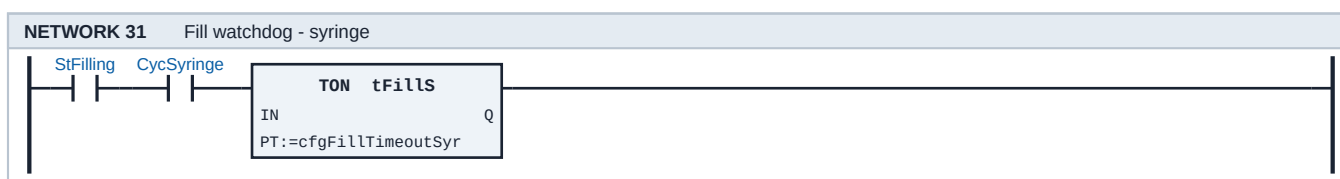
Watchdog on the locking step: StLocking may last at most 2 s. IN drops when the state advances, so the timer self-cancels on success.



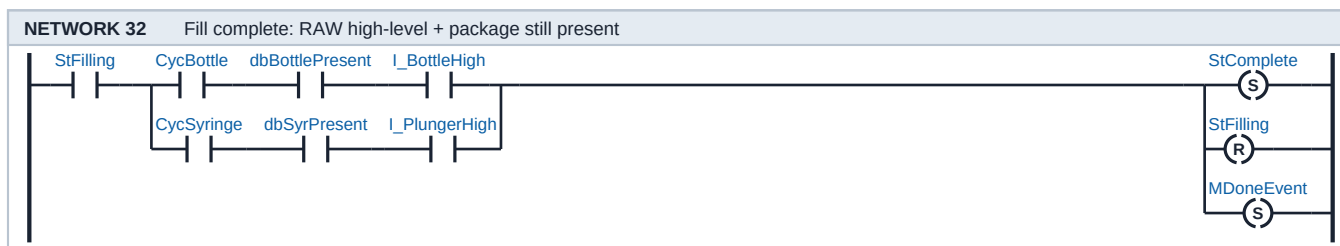
The Locking→Filling transition: door still closed AND the lock bolt confirmed. SET the next state, RESET the current — the transition idiom used by every rung in this section.



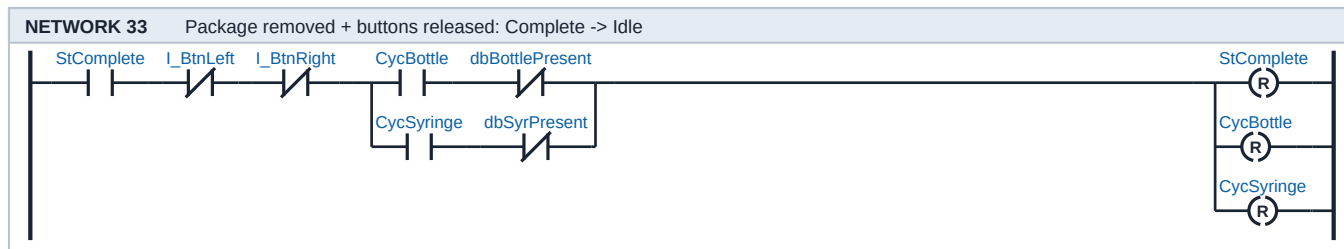
Bottle fill watchdog: 30 s of StFilling without reaching high level is declared stuck (fault 4, network 39).



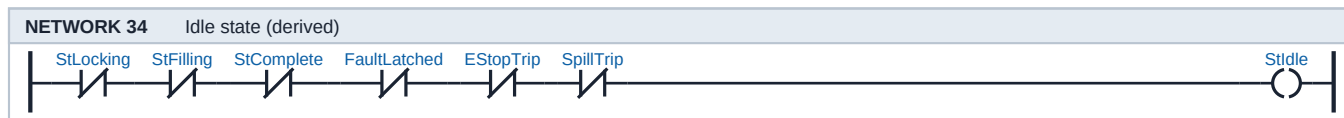
Syringe fill watchdog, 20 s.



Fill complete — the one place raw, un-debounced level sensors are used on purpose: the state follows the very scan the sensor trips, and the valve rungs (45/46) cut off on the same raw signal. The debounced PRESENCE contact in each branch stops a sensor glint during a mid-fill removal from masquerading as success (presence drops instantly; fault 7 wins instead). MDoneEvent hands the completion to the counters even if a trip clears StComplete this same scan.



Cycle closure: buttons released (anti-repeat across cycles) AND the filled package actually removed; the cycle registration bits clear with it. If the done lamp never clears, one of these contacts is the reason — usually a presence sensor still seeing residue.



IDLE is derived, not stored: 'no cycle state and no trip'. It cannot get out of sync, because it is recomputed from the truth every scan.

Section H · Fault capture (first-out) & cycle abort (networks 35–44)

Principle: **first-out fault capture**. When something breaks, several symptoms usually follow within milliseconds — the door pops, the level drops, the watchdog expires. What the troubleshooter needs is the FIRST domino. Each fault rung writes its code only if FaultCode is still 0 (the EQ-then-MOVE chain), so the register preserves the original cause until reset. Section order inside H is deliberate: E-stop and spill first — if the E-stop caused the door signal to drop, code 1 wins the race in the same scan. Network 43 sits AFTER every transition rung of section G so that on any trip, its R coils win the scan and the cycle dies immediately.



First-out capture, E-stop: if FaultCode is still 0, write 1. E-stop is checked FIRST in section H so that when one root cause knocks over several signals in the same scan, code 1 wins the race.



Spill is second in the pecking order.



Door integrity during a cycle: open at any cycle stage, or unlocked while actually filling. The three branches distinguish which combination occurred — online you can see which one carried power.



Lock watchdog expiry → code 8. Distinct from code 2 (door opened) because the remedy is different: 8 is 'lock never engaged' (solenoid/bolt/feedback), 2 is 'door integrity lost afterwards'.



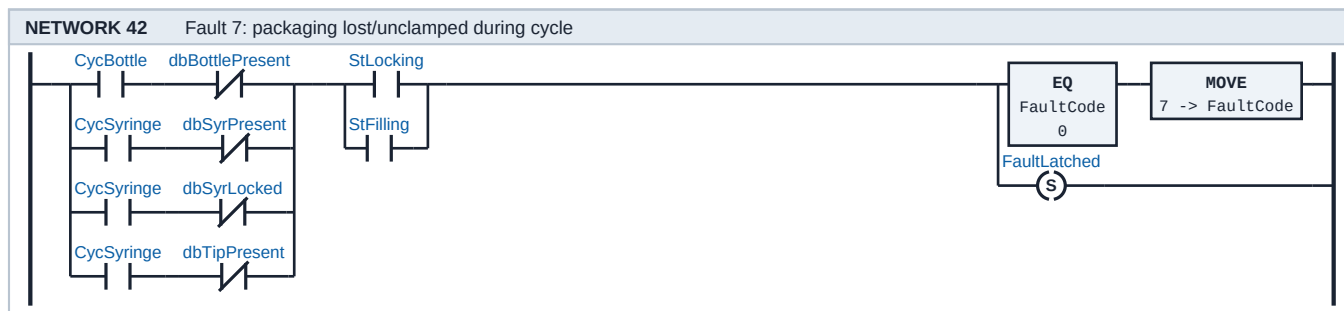
Either fill watchdog → code 4. See the fault table in chapter 6 for the cause list — supply, clog, valve, or a blind level sensor.



A plausibility error DURING a cycle is promoted from warning to fault: mid-fill the sensors are load-bearing; the machine must not keep pouring on data it cannot trust.



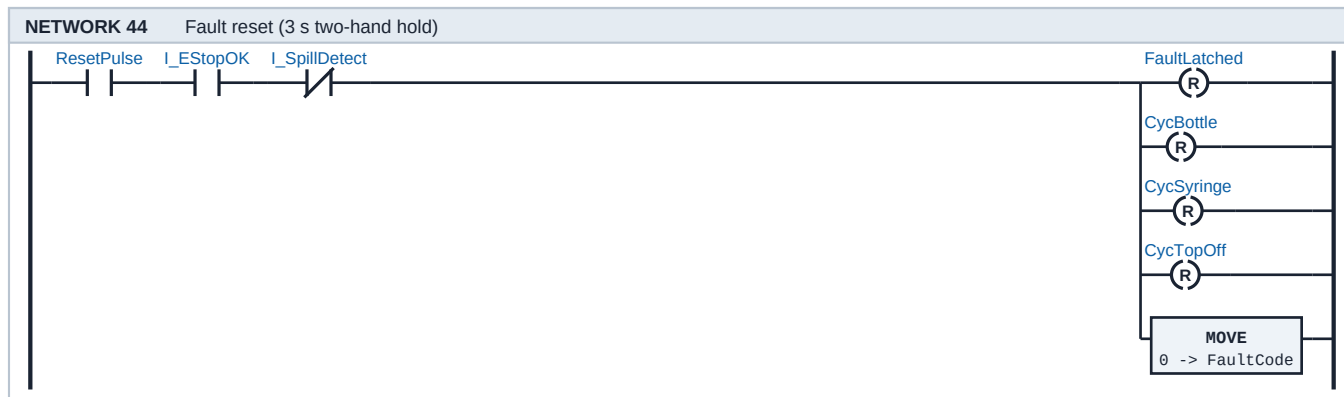
The registered cycle type disagrees with the live mode sensor → the fixture signal was lost mid-cycle. Because the valves follow Cyc* (registration), the wrong valve can never open — the cycle just aborts cleanly.



Package integrity during the cycle: bottle gone, or syringe gone/unclamped/tip lost. Debounced bits drop instantly on removal (network 03 note), so the abort is immediate.



The abort rung — placed AFTER every set/transition rung of section G, so when an S and an R coil fire in the same scan, this R executes last and wins. One rung guarantees: any trip, cycle dead, this very scan.



The common reset: the release-then-hold gesture (network 15) plus proof the E-stop is healthy and the tray is dry. Clears the flag, the cycle registration and the stored code. The E-stop/spill latches ALSO check their own cause (16/18) — belt and braces. Other causes (door, timeout...) may be acknowledged before the cause is fixed; PermStart blocks any restart until it actually is.

Section I · Outputs (networks 45–54)

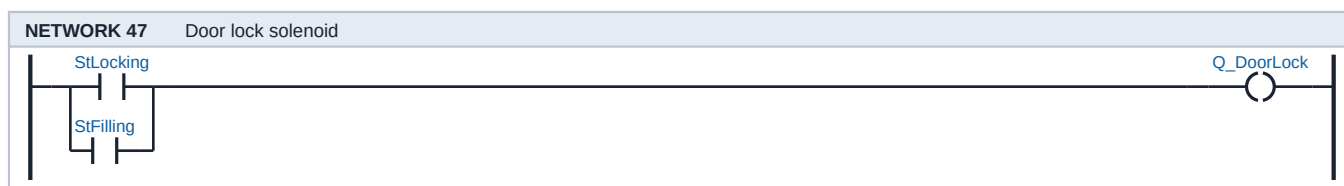
Principle: **defence in depth at the outputs**. The valve rungs re-check the entire safety chain (SafetyOK = raw E-stop input AND no latched E-stop/spill/fault trip), door closed, door locked and the high-level sensor in series with the state machine's own permission. Logically redundant — the state machine already guarantees them — but if any rung above is ever edited carelessly, the output rung still cannot energise a valve with the door open, a trip latched, or the package already full. Cheap insurance at the only place where software touches something hazardous. The lamp rungs then simply publish internal state, and the filling lamp deliberately mirrors the actual valve COMMANDS, not the state bit, so lamp and solenoid can never disagree.



The bottle valve: state machine permission AND the full safety chain (SafetyOK re-checks the raw E-stop input and every latched trip) AND the door/level interlocks re-read raw. Even a corrupted state bit cannot open this valve with the door unlocked or a trip latched.



Syringe valve, same construction.



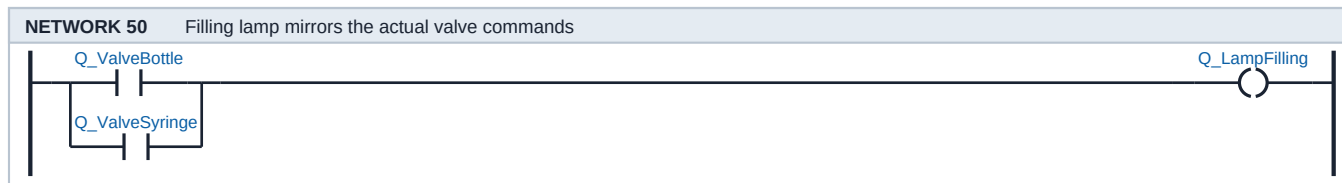
Door locked exactly while a cycle needs it (LOCKING + FILLING). Idle, Complete and Fault states leave the door free — an operator is never locked out of a faulted machine. Energise-to-lock: power loss unlocks the guard, acceptable only because the valves are spring-return closed (see GVL note).



Mode lamp, bottle. Both mode lamps off = mode signal absent/settling (see network 08 note).



Mode lamp, syringe.



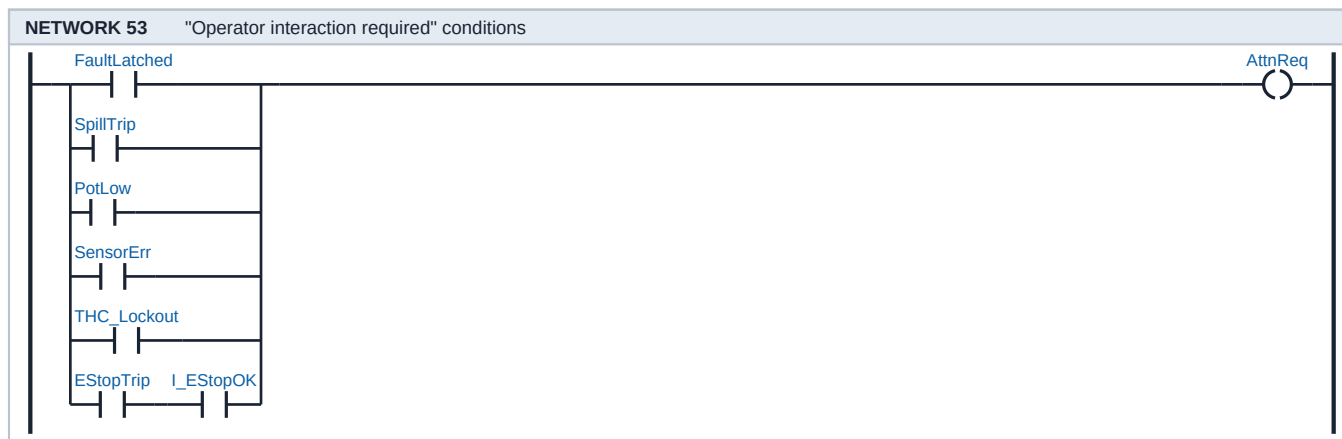
The filling lamp mirrors the valve **COMMANDS**, not **StFilling** — so during the brief **LOCKING** step the lamp is off, and lamp/solenoid can never contradict each other. Lamp off + glue flowing = valve mechanically stuck: hardware.



Done lamp = **StComplete**, held until the package is removed (network 33).



E-stop lamp is **STEADY** (distinct from the flashing attention lamp): on while pressed **OR** while the trip awaits reset. Lamp on with the button released = reset gesture still owed.



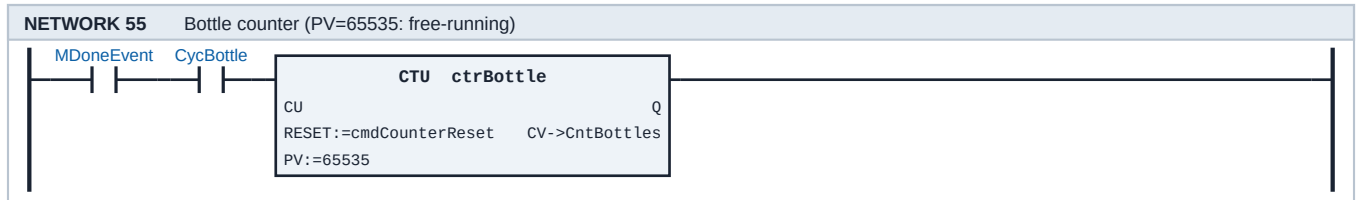
Everything that needs a human collects here: latched faults, spill, low pot, sensor errors, two-hand lockout, and 'E-stop released but not reset'. One variable to test when asking 'why is the attention lamp flashing?' — check its six branches online.



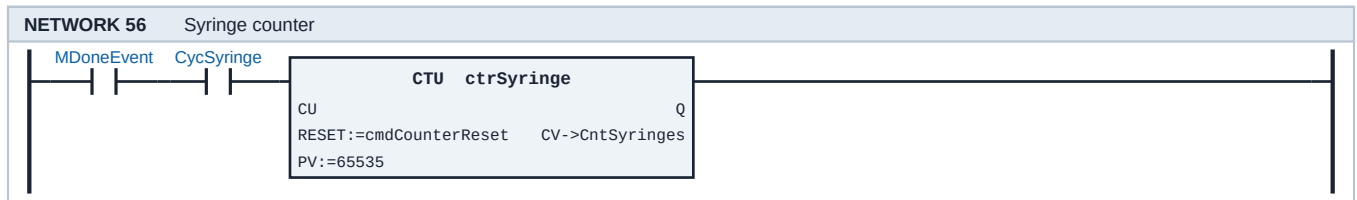
AND with **Blink** turns the steady condition into the specified flashing lamp. If it is **ON** steady, **Blink** froze — see network 01.

Section J · Production counters (networks 55–58)

Principle: **counting events, not conditions** — via a producer/consumer handshake. Network 32 latches **MDoneEvent** the instant a fill completes; the counters consume it here and network 57 clears it, so it lives exactly one scan and each completion counts exactly once — even if a same-scan trip immediately knocks **StComplete** back out (the glue is in the package either way). Aborted cycles never fire the event and are deliberately not counted. **PV** is set to 65535 because the CODESYS CTU stops counting when **CV** reaches **PV** — a classic surprise.



Counts bottle cycles from the one-scan MDoneEvent (latched by network 32, consumed by network 57). PV=65535 because the CODESYS CTU stops counting at PV — with the default 0 it would never count at all. CV is copied (WORD->UINT) to the HMI-visible counter.



Syringe twin of 55.



Consumes the completion event: MDoneEvent lives exactly one scan — set by 32, read by 55/56, cleared here. The same handshake pattern as network 58.



Handshake: the HMI sets cmdCounterReset; the program consumes it (both counters saw it this scan in 55–56) and clears it, so the command acts exactly once.

6 · Troubleshooting guide

6.1 · Fault codes — first-out causes and what to check

Code	Meaning	Likely causes → checks, in order
1	E-stop	Chain open: a pressed button somewhere, a pulled cable, a dead channel. Walk the chain with the safety relay LEDs, then reset (twist out E-stop, hold both buttons 3 s).
2	Door opened / unlocked mid-cycle	Operator opened it; door bounced (watch I_DoorClosed online while wiggling the door); lock solenoid dropped out (wiring, PSU dip).
3	Spill detected	Real leak (inspect nest & lines) or residue bridging the tray probe — CA residue is conductive enough. Clean and dry the tray; reset is refused while it still reads wet.
4	Fill timeout	Pot pressure low/empty (but code 4 with PotLow also flashing points at supply), clogged line/needle — CA cures in moist air, valve not actually opening (listen / watch Q_LampFilling), or a high-level sensor that cannot see the glue (misaligned, fouled). Increase cfgFillTimeout* only after ruling those out.
5	Sensor plausibility	Two sensors disagree with physics — see networks 19–20 for exactly which combinations. Clean lenses (CA fogs optics!), re-align, check for a package half-inserted.
6	Mode signal lost mid-cycle	Fixture ID sensor lost its target: fixture not seated, sensor gap drifted, cable strain. Watch I_BottleMode online while pressing on the fixture.
7	Package lost / unclamped mid-cycle	Bottle lifted by line pressure, syringe clamp creeping open, or a presence sensor dropping out from vibration — watch the db* bits during a fill.
8	Door lock failed to engage	Bolt blocked (door not fully closed), solenoid dead, feedback switch dead. You have 2 s (cfgLockTimeout) — watch I_DoorLocked respond to Q_DoorLock online.

6.2 · Symptoms that do not set a fault code

Symptom	Where to look
Nothing happens on two-hand press	Online: is StartPulse firing? If TwoHandOK never goes TRUE → THC_Lockout stuck (release BOTH buttons fully) or one button dead (networks 10–13). If TwoHandOK fires but no start → walk the PermStart chain (network 25) contact by contact: the first open contact is your answer — SafetyOK? door? PotLow? StIdle? ReadyBottle/ReadySyringe (network 22/23)?
Attention lamp flashing, no fault code	The lamp also flashes for live conditions that are not latched faults: PotLow (refill the pot), SensorErr (networks 19–21 — which impossible combination is active right now?), THC_Lockout (release both buttons), or E-stop released but not yet reset.
Mode lamps both OFF	Fixture signal is mid-change or the debounce (200 ms) is still running — if permanent, the fixture sensor is not seeing its target (network 07/08).
Done lamp never clears	Network 33 needs BOTH buttons released AND the package actually removed — a presence sensor still TRUE (residue, reflective nest) keeps StComplete latched.
Valve closes but lamp says filling	Q_LampFilling mirrors the COMMAND (network 50). If the lamp is off and glue still flows, the valve is mechanically stuck open — hardware, not logic.
Counters not counting	The counters consume the one-scan MDoneEvent latched by network 32 — a cycle aborted by a fault never completes and is deliberately not counted. Watch MDoneEvent with a breakpoint on network 55 if in doubt; it is already FALSE again by the end of every scan (network 57).

6.3 · Diagnosing online with CODESYS

Login → watch the power flow. Open PLC_PRG in the online view: TRUE contacts and energised rung segments are highlighted blue. Find the leftmost cold contact in the rung that should be firing — that contact IS the diagnosis in 90 % of cases.

Build a watch list with StIdle...StComplete, CycBottle/CycSyringe, FaultCode, PermStart, ReadyBottle, ReadySyringe, TwoHandOK. One glance tells you where the sequence stopped.

Timers tell on themselves: click a TON online and read ET. A watchdog at 1.9 s of a 2 s preset is about to trip; a debounce that never accumulates means its input is chattering.

Force with discipline. Forcing dbBottlePresent TRUE to test the sequence is fine with the door open and the pot depressurised — but never force a valve output, and always use 'Unforce all' before leaving. The safety layer will stop the machine, but you will chase ghost faults.

The first-out code beats the alarm flood: when several things look wrong, trust FaultCode — everything after the first cause is usually consequence, not cause.

7 · Commissioning & test checklist

Run this top to bottom with the pot DEPRESSURISED and disconnected from glue until step 6. Tick each line; every line maps to the network it proves.

#	Test	Pass criterion	Proves
1	Wiggle-test every input in the I/O mapping view (door, sensors, buttons, E-stop)	each toggles cleanly, correct polarity per §2.1	wiring & addresses
2	Both mode fixtures in turn	correct mode lamp within 0.2 s; both lamps off mid-swap	07–08, 48–49
3	Press one button, wait, add the second	no start; attention lamp flashes until both released	10–14, 53
4	Two-hand press with door open	no start (PermStart cold at I_DoorClosed)	25
5	Valid start, dry	lock engages < 2 s, StFilling reached, valve output LED on	26–29, 45–47
6	Wet fill, bottle	valve closes the instant I_BottleHigh trips; no overshoot past the fill line	32, 45
7	Open door mid-fill (dry!)	instant abort, fault 2, valves off	37, 43
8	Press E-stop mid-fill	instant abort, E-stop lamp steady, fault 1; reset needs release + 3 s hold	16, 35, 44
9	Block the high-level sensor and start (dry)	watchdog aborts at cfgFillTimeout*, fault 4	30–31, 39
10	Wet the drip tray probe	fault 3; reset refused until dried	17–18, 36
11	Cover a level sensor with no bottle present	SensorErr after 0.5 s, start refused, attention flashes	19–21
12	Remove bottle mid-fill (dry)	instant abort, fault 7	42–43
13	Full bottle in nest, try to start	refused (ReadyBottle cold at I_BottleHigh)	22
14	Empty the pot below the level sensor	attention flashes after 2 s; running fill completes; next start refused	09, 25
15	Ten consecutive good fills of each type	counters advance by exactly ten each	55–57

Timing parameters

Tuning: cfgFillTimeoutBottle/Syr should be $\sim 1.5 \times$ your slowest legitimate fill (measure ten). cfgDebPresence 100 ms suits mechanical bounce; raise cfgDebSensorErr if load/unload transients ever annunciate. Keep cfgSyncWindow at 500 ms — it matches the hardware two-hand relay.

Appendix A · Variable cross-reference

Generated automatically from the network data. 'Written in' = the network that drives the variable (coil / FB instance / MOVE target); 'read in' = every network with a contact or operand on it. When a variable misbehaves, these are the only rungs that can be responsible.

Variable	Kind	Written in	Read in
AttnReq	internal	53	54
Blink	internal	02	54
BottleSensErr	internal	19	21
cmdCounterReset	internal	58	58
ctrBottle	timer/FB	55	—
ctrSyringe	timer/FB	56	—
CycBottle	internal	27, 33, 44	30, 32, 33, 41, 42, 45, 55
CycSyringe	internal	27, 33, 44	31, 32, 33, 41, 42, 46, 56
CycTopOff	internal	27, 44	—
dbBottlePresent	internal	03	19, 22, 32, 33, 42
dbSyrLocked	internal	05	20, 23, 42
dbSyrPresent	internal	04	20, 23, 32, 33, 42
dbTipPresent	internal	06	20, 23, 42
ESTopTrip	internal	16	15, 16, 24, 34, 35, 43, 52, 53
FaultCode	internal	35, 36, 37, 38, 39, 40, 41, 42, 44	35, 36, 37, 38, 39, 40, 41, 42
FaultLatched	internal	35, 36, 37, 38, 39, 40, 41, 42, 44	15, 24, 34, 43, 53
ftTwoHand	internal	12	—
I_BottleHigh	physical input	—	19, 22, 32, 45
I_BottleLow	physical input	—	19, 27
I_BottleMode	physical input	—	07, 08
I_BottlePresent	physical input	—	03
I_BtnLeft	physical input	—	10, 11, 12, 13, 15, 33
I_BtnRight	physical input	—	10, 11, 12, 13, 15, 33
I_DoorClosed	physical input	—	25, 29, 37, 45, 46
I_DoorLocked	physical input	—	29, 37, 45, 46
I_ESTopOK	physical input	—	16, 24, 44, 52, 53
I_PlungerHigh	physical input	—	20, 23, 32, 46
I_PlungerLow	physical input	—	20, 27
I_PotLevelOK	physical input	—	09
I_SpillDetect	physical input	—	17, 18, 44
I_SyrLocked	physical input	—	05
I_SyrPresent	physical input	—	04
I_TipPresent	physical input	—	06
MDoneEvent	internal	32, 57	55, 56, 57
ModeBottle	internal	07	22, 41, 48

Variable	Kind	Written in	Read in
ModeSyringe	internal	08	23, 41, 49
PermStart	internal	25	26, 27
PotLow	internal	09	25, 53
Q_DoorLock	physical output	47	—
Q_LampAttention	physical output	54	—
Q_LampBottle	physical output	48	—
Q_LampDone	physical output	51	—
Q_LampEStop	physical output	52	—
Q_LampFilling	physical output	50	—
Q_LampSyringe	physical output	49	—
Q_ValveBottle	physical output	45	50
Q_ValveSyringe	physical output	46	50
ReadyBottle	internal	22	25, 27
ReadySyringe	internal	23	25, 27
ResetArmed	internal	15	15
ResetPulse	internal	15	16, 18, 44
SafetyOK	internal	24	25, 45, 46
SensorErr	internal	21	22, 23, 40, 53
SpillTrip	internal	18	15, 18, 24, 34, 36, 43, 53
StartPulse	internal	14	26, 27
StComplete	internal	32, 33, 43	33, 34, 51
StFilling	internal	29, 32, 43	30, 31, 32, 34, 37, 40, 41, 42, 45, 46, 47
StIdle	internal	34	25
StLocking	internal	26, 29, 43	28, 29, 34, 37, 40, 41, 42, 47
SyrSensErr	internal	20	21
tBlinkA	timer/FB	01	02
tBlinkB	timer/FB	02	01
tDbBerr	timer/FB	19	—
tDbBotPres	timer/FB	03	—
tDbModeB	timer/FB	07	—
tDbModeS	timer/FB	08	—
tDbPotLow	timer/FB	09	—
tDbSerr	timer/FB	20	—
tDbSpill	timer/FB	17	18
tDbSyrLock	timer/FB	05	—
tDbSyrPres	timer/FB	04	—
tDbTipPres	timer/FB	06	—
tDiscl	timer/FB	10	12
tDiscR	timer/FB	11	12
tFillB	timer/FB	30	39

Variable	Kind	Written in	Read in
tFills	timer/FB	31	39
THC_Lockout	internal	12	12, 13, 53
tLockT0	timer/FB	28	38
tResetHold	timer/FB	15	—
trigReset	timer/FB	15	—
trigStart	timer/FB	14	—
TwoHandOK	internal	13	12, 14, 15

Appendix B · Importing the program into CODESYS

1. Create a Standard project (CODESYS 3.5, your target device), language of PLC_PRG: **Ladder Logic Diagram (LD)**.
2. Add a Global Variable List named **GVL_IO** (right-click Application → Add Object) and paste the contents of GVL_IO.txt into it.
3. Copy the VAR...END_VAR block from the top of PLC_PRG_Ladder.txt into PLC_PRG's declaration editor.
4. Transcribe the 58 networks IN ORDER from PLC_PRG_Ladder.txt / chapter 5. Insert contacts and coils with the toolbar; timers, triggers and counters via 'Insert Empty Box' and typing TON / R_TRIG / CTU; EQ and MOVE via the same box mechanism. Assign S/R modifiers by right-clicking a coil.
5. Alternatively, create PLC_PRG as an **ST** POU and paste PLC_PRG.st — it is a 1:1 mirror of the ladder and behaves identically. Many people commission in ST, then transcribe to LD for the maintenance team once the logic is proven.
6. Map the %IX/%QX addresses to your hardware in the device I/O mapping editor.
7. Use simulation mode (Online → Simulation) plus the watch list from §6.3 to dry-run the whole sequence before touching hardware — every scenario in chapter 7 can be rehearsed by writing the input variables by hand.

File manifest

Companion files in this folder: **GVL_IO.txt** (all variables + wiring notes) · **PLC_PRG_Ladder.txt** (the canonical rung listing) · **PLC_PRG.st** (Structured Text mirror for simulation) · **tools/** (the scripts that generated this document).